

## SCS 3204: AUTOMATA THEORY

### Lecture VI: Finite Automata – More Examples

#### Definition

- A finite automaton (FA) consists of:
  - A finite set of states one of which is the **start state** and some (maybe none) are **final states**
  - An alphabet  $\Sigma$  of possible **input** letters that form input strings that are read one letter at a time
  - A finite set of **transitions** that tell for each state and for each letter of the input alphabet, which state to go to next.

#### How it works

- Presented with an input string at start state
- Reads input string letter by letter beginning with the leftmost letter
- Each letter read causes the automaton to transit to a new state (where the next letter will be read)
- The sequence ends when the last input letter has been read
- If the end occurs at a final state then the input string has been accepted (recognized) by the automaton

#### Example

- Construct a FA that will accept only those strings defined over the alphabet  $\{0, 1\}$  that have a **1** as the fourth letter and reject all other words.

#### Example

- Construct a FA that accepts exactly the three strings **aba**, **bab** and **ba**.

#### Example: EVEN-EVEN

- Describe a computer algorithm that accepts a long string of *a*'s and *b*'s and determines whether the number of *a*'s is even and the number of *b*'s is even.

### **EVEN-EVEN – an alternative algorithm**

- Define a RE for Language EVEN-EVEN ( $E$ )

$$E = [aa + bb + (ab + ba)(aa + bb)^*(ab + ba)]^*$$

- This regular expression represents the collection of all words that are made up of “syllables” of three types:

$$\text{type}_1 = aa$$

$$\text{type}_2 = bb$$

$$\text{type}_3 = (ab + ba)(aa + bb)^*(ab + ba)$$

$$E = [\text{type}_1 + \text{type}_2 + \text{type}_3]^*$$

### **EVEN-EVEN – an alternative algorithm**

- To determine if a string is in  $E$  using this definition, we proceed as follows:
- Set `even_flag` (counter) to 0
- Read in two letters at a time. If they are the same i.e. `type1` or `type2`, read the next two.
- If they are of `type3`, reverse `even_flag`.
- At the end of the string, if `even_flag` = 0, then the input string contains an even number of a's and b's, else it does not.
- Consider the input string  
(aa)(ab)(bb)(ba)(ab)(bb)(bb)(aa)(ab)(ba)(aa)(ab)(bb)  
`even_flag` is reversed 6 times and ends at 0.

### **A FA for EVEN-EVEN**

- EVEN-EVEN = { $\Lambda$  aa bb aaaa aabb abab abba baab baba bbaa bbbb aaaaaa aaaabb aaabab .....}
- A FA for EVEN-EVEN:

### **Example**

- Consider the language  $L$  defined over the alphabet  $\Sigma = \{a, b\}$ , and which consists of strings that have an even number of letters
- Define a machine that will accept  $L$ .

### **Example**

- Define a FA that accepts all words over the alphabet  $\Sigma = \{a, b\}$  that have different first and last letters i.e. If the word begins with an  $a$ , it must end with a  $b$  to be accepted, and vice versa.

### **Example**

- Consider the language  $L$  defined over the alphabet  $\Sigma = \{a, b\}$ , and which consists of strings that contain a double  $a$ :
- What regular expression defines this language?
- Define a machine that will accept  $L$ .

### Example

- Consider the language L defined over the alphabet  $\Sigma = \{a, b\}$ , and which consists of strings that contain an odd number of a's
- Define a machine that will accept L.
- What regular expression defines this language?

### Example

- Construct a finite automata accepting:
  1. The set of integers
  2. The set of signed integers

### Example

- We are programmers hired to write a spell-checker for mini-English, where mini-English consists of the words *cook*, *read*, *sing*, *eat* and *laugh* and their conjugations for present, progressive and past tense.
- What regular expression defines mini-English?
- Define a FSM that will accept mini-English.